

Quantization, LoRA & QLoRA Explained Simply

A beginner-friendly guide to how large AI models are made smaller, cheaper, and easier to fine-tune — explained with everyday analogies.

- 1 What is quantization, and why does it matter
- 2 Full & half precision, calibration, PTQ vs QAT
- 3 Symmetric vs asymmetric quantization
- 4 LoRA — low-rank adaptation
- 5 QLoRA — quantized LoRA

QUANTIZATION

What is Quantization?

Making the numbers inside an AI model smaller, so the whole model takes up less memory.

Imagine you buy a huge robot brain (that's an LLM — Large Language Model). It's super smart, but it's also **huge** and needs a giant, expensive computer to run. Quantization is one of the main tricks used to make this robot brain smaller and cheaper to run.

Quantization

Conversion from a higher memory format to a lower memory format — i.e. shrinking big, detailed numbers into smaller, lighter ones.

EVERYDAY ANALOGY

Think of saving a photo. A super high-quality photo file might be 50 MB — looks amazing but takes forever to send. A compressed version might be 2 MB — still looks pretty good and sends instantly. You didn't change *what's in* the photo, just how much memory it uses. That is exactly what quantization does to an AI model's numbers.

Weights: the numbers inside the brain

An AI model is built from a network of connected nodes (a **neural network**). Every connection between nodes has a number attached to it that tells the AI "how important" that connection is. All these numbers together are called **weights**, and they are stored as a big grid of numbers — a **matrix**.

Why it matters: FP32 and scale

Normally, each weight is stored as **FP32** — "Floating Point, 32 bits." Think of 32 bits like using 32 tiny on/off switches to describe one single number very precisely. This is called **Full Precision** or **Single Precision**.

SCALE OF THE PROBLEM

A model like **LLaMA 2** has **70 billion** of these weight numbers. If every single number needs 32 bits of storage, the total memory required becomes massive — which is exactly why huge AI models need expensive, powerful GPUs just to run.

Quantization's job is to shrink each number from a big format (e.g. 32-bit) down to a smaller format (e.g. 8-bit or even 4-bit), so the whole model becomes lighter and can run on cheaper, smaller computers — just like turning a 50 MB photo into a 2 MB photo.

QUANTIZATION

The Three Building Blocks

Precision formats, calibration, and the two modes of quantization.

1. Full precision vs half precision

Full precision (FP32)

Uses big, detailed 32-bit numbers. Very accurate, but heavy on memory and slow to compute.

Half precision (FP16)

Uses smaller 16-bit numbers. Slightly less detailed, but much lighter and faster to run.

2. Calibration → model quantization

Calibration is like "practicing" or "checking" the best way to shrink the numbers without breaking the AI's intelligence. Before fully compressing the model, it's tested on sample data to find the smartest way to compress the weights while losing as little accuracy as possible.

3. Modes of quantization

There are two main strategies for when the shrinking happens:

Modes of
quantization



Post training
quantization

Quantization
aware training

Post training quantization (PTQ)

Train the model fully at full size first. *After* training is done, shrink the numbers down. Like baking a full cake, then slicing it smaller to fit a lunchbox.

Quantization aware training (QAT)

Teach the model to get used to working with smaller numbers *while* it is still learning — so it becomes smart and small right from the start.

Symmetric vs asymmetric quantization

These are two different ways to actually squeeze a wide range of numbers into a smaller range.

Symmetric quantization

Like a perfectly balanced seesaw. Numbers are treated as spreading equally in the positive and negative direction, so **zero always sits exactly in the middle**. Simple and fast to compute.

Asymmetric quantization

Like an unbalanced seesaw. When data isn't evenly spread around zero (e.g. only positive numbers), zero shifts using a **"zero-point"** correction — making better use of the available range.

EVERYDAY ANALOGY

Imagine dividing a pizza. **Symmetric** = always cutting it perfectly in half, no matter what. **Asymmetric** = cutting it based on how hungry each person actually is, because that fits reality better.

Aspect	Symmetric	Asymmetric
Zero position	Always centered	Shifts, using a zero-point
Best for	Data balanced around zero	Uneven data (e.g. only positive)
Complexity	Simpler, faster	Slightly more complex
Common use	Weights	Activations (e.g. after ReLU)

SECTION SUMMARY

Quantization shrinks large, precise numbers into smaller, lighter formats. It can happen after training (PTQ) or during training (QAT), and the shrinking itself can be balanced around zero (symmetric) or shifted to fit the data better (asymmetric).

LORA

LoRA — Low-Rank Adaptation

Teaching a giant AI model something new, without rewriting the whole thing.

The problem with full fine-tuning

Imagine the AI robot brain is like a **giant textbook** with 70 billion facts memorized in it. Now say you want to teach it something new — maybe how to talk like a doctor, or answer questions about your school's rules.

The old-school way, **full fine-tuning**, means opening up the entire textbook and rewriting almost every page — touching all 70 billion numbers. That needs a massive, expensive computer, like renting a whole stadium just to write one page in a notebook.

The LoRA idea

LoRA (Low-Rank Adaptation)

Instead of rewriting the whole textbook, add a few small "sticky notes" of new information on top of specific pages — while the original textbook stays frozen and untouched.

- The original AI brain stays **frozen** (locked, unchanged).
- Only a small, extra set of numbers — tiny "sticky notes" — sits on top and gets trained.
- These sticky notes are far smaller and lighter than the whole textbook, needing much less computer power.

How it works

Weights are stored as big matrices (grids of numbers). In LoRA, instead of changing the giant grid directly, we create **two much smaller grids** that, when multiplied together, approximate the change we want to make — similar to storing a small sketch that, combined with the original city map, is enough to find your way around the new parts.

Why LoRA is awesome

Faster & cheaper

Only a tiny part is adjusted, not everything — so training needs far less memory and time.

Small & reusable

The "sticky note" files are tiny, and you can swap different sets for different tasks on the same base model.

QLORA

QLoRA — Quantized LoRA

Combining quantization and LoRA to fine-tune huge models on ordinary hardware.

QLoRA

First shrink the giant original AI brain using quantization, then add the small trainable LoRA "sticky notes" on top of the shrunken brain.

- 1 **Quantization step:** shrink the giant original AI brain into a smaller memory format (e.g. squeeze 32-bit numbers into 4-bit numbers).
- 2 **LoRA step:** add small trainable "sticky notes" on top of this shrunken, frozen brain to teach it new things.

EVERYDAY ANALOGY

Imagine a huge, heavy textbook. **LoRA** keeps the textbook as-is, and just clips lightweight sticky notes on top. **QLoRA** first shrinks the textbook into a compressed, pocket-sized version, and *then* clips the same lightweight sticky notes on top of that.

Because of this combination, QLoRA needs **even less** computer power than regular LoRA — the base brain itself is smaller (thanks to quantization), and only small sticky notes are trained on top (thanks to LoRA). This is why, with QLoRA, even a normal laptop or a single decent GPU can fine-tune huge models like LLaMA 2 (70 billion parameters).

Visual summary



QLoRA = quantize the frozen base model first, then train small LoRA layers on top of it.

Full fine-tuning vs LoRA vs QLoRA

Aspect	Full fine-tuning	LoRA	QLoRA
What changes	Entire model (all weights)	Small extra "sticky note" layers	Small extra layers + shrunken model
Memory needed	Very high	Much less	Even less
Speed	Slow	Fast	Fast
Original model	Modified everywhere	Frozen (untouched)	Frozen and compressed
Good for	Teams with supercomputers	Regular developers	A single GPU or laptop

ONE-LINE TAKEAWAY

LoRA = don't rewrite the whole book, just add small sticky notes of new knowledge on top.
QLoRA = shrink the whole book first to save space, then add your sticky notes — making fine-tuning even lighter and cheaper.